

CORE QUANT MATH

# Calculus Foundations for Market Models

A beginner field guide to derivatives, integrals, optimization, sensitivity, and continuous-growth thinking for quant research systems.

## Manual profile

Category: Calculus

Difficulty: Beginner

Format: Single-column stacked reference-card manual

Purpose: Fast recall, implementation intuition, and market-model connections

# How to Use This Manual

BEGINNER

**STUDY PATH**

Read the pages in order once, then use the title bars and related terms as a lookup system. Each card is designed to stand alone but also connect to the larger research pipeline.

**READING METHOD**

For each concept, identify the input, output, local change, accumulation, or optimization target. That one habit turns calculus from abstract math into model-debugging language.

**PRACTICE METHOD**

Recreate the formulas with tiny numeric examples. Then implement the same idea with a simple Python function, a pandas Series, or a small parameter grid.

**QUANT RESEARCH FOCUS**

The goal is not symbolic perfection. The goal is knowing when a slope, curvature, integral, gradient, or approximation is valid enough to help a trading or risk system.

**USE WITH OTHER VOLUMES**

Pair this manual with functions, logarithms, sequences, probability, statistics, optimization, and backtesting. Calculus sits between raw formulas and dynamic model behavior.

**WARNING LABEL**

Market data is sampled, noisy, regime-dependent, and full of jumps. Calculus gives clean language, but every assumption must be validated against actual data and execution rules.

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Table of Contents I

BEGINNER

| Concept                                               | Page |
|-------------------------------------------------------|------|
| Calculus as Local Market Thinking                     | 7    |
| Functions as Market Machines                          | 8    |
| Limits: What Happens As You Get Close                 | 9    |
| Continuity and Price Paths                            | 10   |
| Difference Quotient                                   | 11   |
| Derivative: Instantaneous Rate of Change              | 12   |
| Slope in a Price Series                               | 13   |
| Rate of Change vs Return                              | 14   |
| First Derivative as Momentum Analogy                  | 15   |
| Second Derivative as Acceleration                     | 16   |
| Convexity and Curvature                               | 17   |
| Chain Rule                                            | 18   |
| Product and Quotient Rules                            | 19   |
| Log Derivative and Growth Rate                        | 20   |
| Elasticity                                            | 21   |
| Partial Derivatives                                   | 22   |
| Gradient Vector                                       | 23   |
| Multivariable Market Functions                        | 24   |
| Jacobian for Feature Transforms                       | 25   |
| Hessian Matrix                                        | 26   |
| Optimization: Choosing Inputs to Improve an Objective | 27   |
| Critical Points                                       | 28   |
| Taylor Approximation                                  | 29   |
| Linearization                                         | 30   |
| Quadratic Approximation                               | 31   |
| Integration as Accumulation                           | 32   |
| Riemann Sum                                           | 33   |
| Definite Integral                                     | 34   |

# Table of Contents II

BEGINNER

| Concept                                      | Page |
|----------------------------------------------|------|
| Integral of a Return Rate                    | 35   |
| Area Under Curve Metrics                     | 36   |
| Fundamental Theorem of Calculus              | 37   |
| Continuous Compounding Differential Equation | 38   |
| Differential Equations                       | 39   |
| Geometric Brownian Motion Structure          | 40   |
| Drift and Diffusion                          | 41   |
| Sensitivity Greeks: Delta                    | 42   |
| Gamma and Convex Option Exposure             | 43   |
| Vega, Theta, and Rho                         | 44   |
| Risk Exposure as a Gradient                  | 45   |
| Marginal Contribution to Portfolio Risk      | 46   |
| Loss Functions                               | 47   |
| Gradient Descent                             | 48   |
| Learning Rate and Step Size                  | 49   |
| Numerical Derivatives                        | 50   |
| Finite Differences                           | 51   |
| Discrete vs Continuous Models                | 52   |
| Smoothness Assumptions                       | 53   |
| Non-Differentiable Market Features           | 54   |
| Noise and Derivatives                        | 55   |
| Smoothing and Regularization                 | 56   |
| Indicator Derivatives                        | 57   |
| Feature Engineering with Slopes              | 58   |
| Calibration Objective Functions              | 59   |
| Backtest Metrics as Functions                | 60   |
| Drawdown as a Path Function                  | 61   |
| Constraints in Optimization                  | 62   |

# Notation Reference

PURPOSE

Use this page as the symbol decoder for the rest of the manual. The same calculus pattern repeats: define a function, perturb an input, observe an output change, then decide whether the local approximation is trustworthy.

CORE NOTATION TABLE

| Item             | Use                           | Watch for                   |
|------------------|-------------------------------|-----------------------------|
| $P_t$            | price at time $t$             | adjust for splits/dividends |
| $r_t$            | return at time $t$            | simple vs log return        |
| $V(t)$           | portfolio value function      | positive for log growth     |
| $f(x)$           | generic model function        | define input/output units   |
| $f$ prime        | first derivative / slope      | local, not global           |
| $f$ double-prime | second derivative / curvature | noise-sensitive             |
| $\text{grad } f$ | vector of partial derivatives | local direction             |
| $\Sigma$         | covariance matrix             | regime instability          |

MODEL NOTATION TABLE

| Item        | Use                     | Watch for             |
|-------------|-------------------------|-----------------------|
| $w$         | portfolio weights       | constraints matter    |
| $\theta$    | model parameters        | overfit risk          |
| $J(\theta)$ | loss/objective function | metric design matters |
| $dt$        | small time step         | annualization units   |

UNIT DISCIPLINE

Always label whether a quantity is dollars, percent, log return, annualized rate, per-day rate, per-bar change, or volatility. Many calculus mistakes in finance are unit mistakes disguised as formulas.

# Calculus in the Quant Research Pipeline

BEGINNER

**PIPELINE MAP**

Raw market data -> cleaned series -> returns -> features -> signals -> positions -> PnL -> risk metrics -> validation -> deployment.

**WHERE DERIVATIVES APPEAR**

Derivatives appear as slopes, sensitivities, Greeks, gradients, and marginal risk contributions. They answer how a model output changes when an input changes slightly.

**WHERE INTEGRALS APPEAR**

Integrals appear as accumulated returns, expected values, total exposure, realized variance, area-under-curve metrics, and continuous compounding.

**WHERE OPTIMIZATION APPEARS**

Optimization appears when fitting models, calibrating parameters, choosing weights, and tuning strategy rules. Calculus supplies gradient and curvature intuition.

**SYSTEM DESIGN RULE**

Every stage should expose inputs, outputs, assumptions, and sensitivity diagnostics. The cleaner the function boundaries, the easier the research system is to debug.

**VALIDATION CHECKPOINT**

A calculus-based metric is useful only if it survives realistic data handling, transaction costs, slippage, execution timing, and out-of-sample testing.

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Calculus as Local Market Thinking

BEGINNER

## ONE-SENTENCE MEANING

Calculus studies how quantities change, accumulate, and respond to small moves.

## MEMORY JOGGER

Zoom in until a curve looks almost straight; then measure the local behavior.

## PLAIN-ENGLISH CORE

Market models often turn messy time series into functions: price as a function of time, portfolio value as a function of weights, option value as a function of price, or loss as a function of parameters. Calculus gives language for local slope, curvature, area, and sensitivity.

## QUANT USE + MARKET EXAMPLE

Quant use: Use calculus to reason about returns, compounding, optimization, risk exposure, and option sensitivities.

Example: A strategy equity curve can be viewed as  $V(t)$ . Its local slope shows near-term growth rate; its curvature shows whether growth is accelerating or stalling.

## FORMULA INTUITION

Core pattern: input changes a little  $\rightarrow$  output changes a little. Write this as  $\Delta \text{output} / \Delta \text{input}$ , then shrink the input change conceptually.

## PYTHON / SYSTEM IMPLEMENTATION IDEA

Represent model objects as functions:  $\text{feature} = f(\text{raw data})$ ,  $\text{signal} = g(\text{features})$ ,  $\text{portfolio} = h(\text{weights})$ ,  $\text{score} = J(\text{parameters})$ . Then measure changes with finite differences or automatic differentiation.

## SYSTEM FIT, CONFUSIONS, RELATED TERMS

System fit: This page is the bridge between algebraic formulas and model behavior diagnostics.

Confusions to avoid: Calculus is not magic prediction. It explains model response under assumptions; market data can violate smoothness.

Related: functions, limits, derivatives, integrals, optimization, Greeks

## QUICK RECALL

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Functions as Market Machines

BEGINNER

## ONE-SENTENCE MEANING

A function maps an input to an output, like prices to returns or weights to portfolio risk.

## MEMORY JOGGER

Input in, output out. Calculus asks how the output changes when the input nudges.

## PLAIN-ENGLISH CORE

Before derivatives or integrals matter, you need a function. In quant work, functions are everywhere: `return(P_t, P_{t-1})`, `volatility(returns)`, `signal(features)`, `pnl(position, price move)`, and `objective(parameters)`. Calculus turns those machines into measurable objects.

## QUANT USE + MARKET EXAMPLE

Quant use: Define research components clearly so you can test sensitivity, optimize parameters, and debug transformations.

Example: A volatility estimator is a function taking recent returns as input and returning a risk estimate.

## FORMULA INTUITION

If  $y = f(x)$ , then  $x$  is the input and  $y$  is the output. Calculus studies  $dy/dx$ , the local output change per unit input change.

## PYTHON / SYSTEM IMPLEMENTATION IDEA

Write pure functions in Python where possible: inputs are arrays/dataframes, outputs are deterministic arrays/scores. Pure functions are easier to test and differentiate numerically.

## SYSTEM FIT, CONFUSIONS, RELATED TERMS

System fit: Every feature, signal, risk model, and scoring metric should be traceable as a function in the research pipeline.

Confusions to avoid: Do not confuse a function with a time series. A time series is data; a function is a rule applied to data.

Related: domain, range, transformations, feature engineering, objective function

## QUICK RECALL

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.



# Limits: What Happens As You Get Close

BEGINNER

**ONE-SENTENCE MEANING**

A limit describes what a function approaches as the input moves near a value.

**MEMORY JOGGER**

Do not ask only what happens at the point; ask what happens as you approach it.

**PLAIN-ENGLISH CORE**

Limits let calculus talk about local behavior even when direct substitution is awkward. In finance, limits appear when moving from discrete compounding to continuous compounding, from finite differences to derivatives, and from sums to integrals.

**QUANT USE + MARKET EXAMPLE**

Quant use: Use limits to understand approximations: small returns, short time steps, dense sampling, and continuous-time model assumptions.  
Example: As the sampling interval shrinks, a discrete return model can start to resemble a continuous return-rate model.

**FORMULA INTUITION**

Derivative idea:  $f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$ . The limit is the slope as the step size gets tiny.

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

When testing numerical limits, evaluate several smaller step sizes  $h$  instead of trusting one arbitrary  $h$ . Plot estimate versus  $h$  to detect instability.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Limits justify approximations used in compounding, option models, and differential equations.

Confusions to avoid: A limit is not always the actual value at the point. Nearby behavior and point behavior can differ.

Related: continuity, derivative, finite difference, continuous compounding

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Continuity and Price Paths

BEGINNER

**ONE-SENTENCE MEANING**

A continuous function has no sudden jumps, gaps, or breaks.

**MEMORY JOGGER**

You can draw it without lifting the pen. Markets often refuse to be that clean.

**PLAIN-ENGLISH CORE**

Many mathematical models assume continuous paths because calculus works cleanly there. Real price data can gap overnight, jump after news, halt, or trade with discrete ticks. The gap between smooth model assumptions and real market mechanics matters.

**QUANT USE + MARKET EXAMPLE**

Quant use: Check when a model assumes continuity and whether your asset, sampling frequency, and event risk violate it.  
Example: A stock closing at 100 and opening at 92 after earnings is not a smooth path across the overnight interval.

**FORMULA INTUITION**

Continuity at  $a$ : limit as  $x \rightarrow a$  of  $f(x) = f(a)$ . In market terms, the approach value must match the observed value.

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Flag gaps, missing bars, halted periods, and corporate actions before applying smooth estimators or derivative-like features.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Continuity checks belong in data validation before modeling and backtesting.  
Confusions to avoid: Intraday interpolation can make a chart look smooth even when the economic process had jumps.  
Related: limits, jumps, gaps, tick size, data cleaning

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Difference Quotient

BEGINNER

**ONE-SENTENCE MEANING**

A difference quotient measures average change over a finite step.

**MEMORY JOGGER**

Before the derivative, measure rise over run using a real step.

**PLAIN-ENGLISH CORE**

The difference quotient is the practical ancestor of the derivative. In actual market data, you rarely observe true instantaneous changes. You observe sampled prices, then compute finite differences across bars.

**QUANT USE + MARKET EXAMPLE**

Quant use: Use it to estimate slope, return, momentum, and local change from discrete data.

Example: If price moves from 100 to 101 over one bar, the one-bar difference quotient is 1 price unit per bar.

**FORMULA INTUITION**

$[f(x+h) - f(x)] / h$ . If  $h$  is one day, this is a daily average rate of change.

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Use vectorized differences: `delta = series.diff()`; `rate = delta / step_size`. For returns, normalize by price when scale matters.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Difference quotients turn raw prices into change features, which feed signals and diagnostics.

Confusions to avoid: A finite difference is not exactly a derivative unless smoothness and small-step assumptions are acceptable.

Related: derivative, return, momentum, finite difference

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Derivative: Instantaneous Rate of Change

BEGINNER

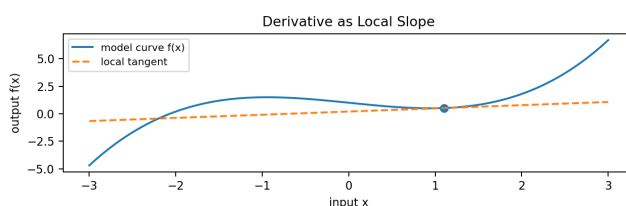
## ONE-SENTENCE MEANING

A derivative is the local slope of a function at a point.

## MEMORY JOGGER

Slope at one exact location, not over the whole chart.

## ANNOTATED CHART



The tangent line is the derivative intuition: local slope at one point, not an average over the whole curve.

## PLAIN-ENGLISH CORE

The derivative answers: if I nudge the input right here, how quickly does the output move? This makes it central for sensitivity analysis, optimization, and local approximations.

## QUANT USE + MARKET EXAMPLE

Quant use: Measure marginal effects such as how portfolio risk changes when a weight changes or how option value changes when the underlying price moves.

Example: An option with delta 0.60 is expected to gain roughly 0.60 dollars for a 1 dollar move in the underlying, locally.

## FORMULA INTUITION

$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$ . The derivative is local, so it can change from point to point.

## PYTHON / SYSTEM IMPLEMENTATION IDEA

Estimate with finite differences when formulas are unavailable:  $\frac{f(x+h)-f(x-h)}{2h}$  is often more stable than forward difference.

## SYSTEM FIT, CONFUSIONS, RELATED TERMS

System fit: Derivatives are the core diagnostic for local model response and parameter sensitivity.

Confusions to avoid: A derivative is not a forecast; it is a local response measure conditional on the model.

Related: slope, tangent line, finite differences, sensitivities

## QUICK RECALL

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Slope in a Price Series

BEGINNER

**ONE-SENTENCE MEANING**

Slope measures the direction and speed of movement in a plotted series.

**MEMORY JOGGER**

Upward slope means rising values; steep slope means fast movement.

**PLAIN-ENGLISH CORE**

For charts, slope is intuitive. For quant research, the issue is scale. A price slope of 1 dollar per day means different things for a 10 dollar asset and a 1000 dollar asset. Returns often fix that comparability problem.

**QUANT USE + MARKET EXAMPLE**

Quant use: Use slope-like features for trend, but normalize when comparing across assets.

Example: A stock rising 1 dollar from 20 to 21 is a 5 percent move; from 200 to 201 it is only 0.5 percent.

**FORMULA INTUITION**

Price slope:  $\Delta P / \Delta t$ . Return slope-like change:  $\Delta P / P$ . Log return:  $\log(P_t / P_{t-1})$ .

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

For cross-sectional models, prefer returns, z-scores, or volatility-scaled slopes over raw dollar slopes.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Slope features belong in feature engineering, but normalization belongs in preprocessing.

Confusions to avoid: Do not compare raw price slopes across assets with very different price levels.

Related: returns, normalization, momentum, trend

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Rate of Change vs Return

BEGINNER

ONE-SENTENCE MEANING

Rate of change is a general change measure; return is a finance-normalized change in value.

MEMORY JOGGER

Slope says how many units; return says how much relative to starting value.

PLAIN-ENGLISH CORE

Calculus often measures  $dy/dx$  in raw units. Finance usually cares about proportional change because capital scales multiplicatively. This is why derivatives, returns, and growth rates must be interpreted with units.

QUANT USE + MARKET EXAMPLE

Quant use: Choose raw change for physical units or price increments; choose return for capital growth and cross-asset comparison.  
Example: A futures tick model may care about raw points; portfolio growth usually cares about percent or log returns.

DIAGNOSTIC MINI-TABLE

| Item              | Use               | Watch for              |
|-------------------|-------------------|------------------------|
| Raw change        | unit movement     | scale mismatch         |
| Return            | capital growth    | compounding convention |
| Derivative        | local sensitivity | smoothness assumption  |
| Finite difference | sampled estimate  | noise and alignment    |

FORMULA INTUITION

Raw change =  $P_t - P_{t-1}$ . Simple return =  $(P_t - P_{t-1}) / P_{t-1}$ . Log return =  $\log(P_t/P_{t-1})$ .

PYTHON / SYSTEM IMPLEMENTATION IDEA

Store both price changes and returns only when each has a purpose. Label units in feature metadata.

SYSTEM FIT, CONFUSIONS, RELATED TERMS

System fit: Unit discipline prevents false signals caused by comparing incompatible scales.  
Confusions to avoid: A derivative can have units; a return is usually dimensionless.  
Related: percent change, log return, units, scaling

QUICK RECALL

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# First Derivative as Momentum Analogy

BEGINNER

**ONE-SENTENCE MEANING**

The first derivative tells whether a function is increasing or decreasing locally.

**MEMORY JOGGER**

Positive slope: moving up. Negative slope: moving down.

**PLAIN-ENGLISH CORE**

Momentum indicators often approximate first-derivative thinking, but they operate on sampled, noisy data. A rising moving average slope or positive recent return is not a mathematical derivative, but it plays a similar local-change role.

**QUANT USE + MARKET EXAMPLE**

Quant use: Use first-change features to describe trend direction, but validate across horizons and regimes.

Example: A 20-day moving average with a positive slope can be a trend filter for long-only exposure.

**FORMULA INTUITION**

If  $f'(x) > 0$ ,  $f$  is locally increasing. If  $f'(x) < 0$ ,  $f$  is locally decreasing.

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Compute rolling regression slope or moving-average difference instead of using one noisy bar difference.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Momentum features are derivative-inspired inputs in signal generation.

Confusions to avoid: Positive recent slope can be noise, not persistent momentum. Use robustness checks.

Related: momentum, trend, moving average slope, finite difference

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Second Derivative as Acceleration

BEGINNER

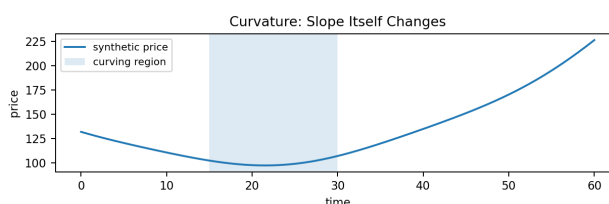
## ONE-SENTENCE MEANING

The second derivative measures how the first derivative is changing.

## MEMORY JOGGER

Slope of the slope.

## SYNTHETIC CHART



Curvature appears when slope changes through time. It can reveal acceleration, deceleration, or nonlinear payoff behavior.

## PLAIN-ENGLISH CORE

If the first derivative is speed, the second derivative is acceleration. In market features, this can describe trend strengthening, trend weakening, or curvature in an equity curve. It is useful but noise-sensitive.

## QUANT USE + MARKET EXAMPLE

Quant use: Use acceleration-like features carefully to detect changing regimes or fading signals.

Example: A strategy equity curve with positive slope but negative curvature may still be profitable while losing momentum.

## FORMULA INTUITION

Second derivative:  $f''(x)$  = derivative of  $f'(x)$ . Finite difference:  $f(x+h) - 2f(x) + f(x-h)$ .

## PYTHON / SYSTEM IMPLEMENTATION IDEA

Estimate second differences on smoothed series, not raw tick noise, unless you specifically model microstructure.

## SYSTEM FIT, CONFUSIONS, RELATED TERMS

System fit: Acceleration diagnostics help identify when a signal or equity curve is changing character.

Confusions to avoid: Second differences amplify noise more than first differences.

Related: curvature, acceleration, convexity, smoothing

## QUICK RECALL

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.



# Convexity and Curvature

BEGINNER

**ONE-SENTENCE MEANING**

Curvature describes how bent a function is; convexity is upward-bending behavior.

**MEMORY JOGGER**

A bowl is convex; an upside-down bowl is concave.

**PLAIN-ENGLISH CORE**

Convexity matters when outcomes are nonlinear. A convex payoff benefits more from favorable moves than it loses from similar unfavorable moves. Option gamma and portfolio drawdown recovery both involve nonlinear curvature ideas.

**QUANT USE + MARKET EXAMPLE**

Quant use: Understand nonlinear payoff profiles, option exposure, and risk functions.

Example: A long call option has positive convexity: gains can accelerate as the underlying rises.

**FORMULA INTUITION**

For one variable,  $f''(x) > 0$  suggests convexity locally;  $f''(x) < 0$  suggests concavity locally.

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Plot payoff diagrams and numerical second differences to see curvature. Use scenario grids rather than only single-point estimates.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Curvature belongs in risk modeling, option analysis, and nonlinear strategy diagnostics.

Confusions to avoid: A strategy with convex-looking backtest returns may still be overfit; curvature does not prove edge.

Related: second derivative, gamma, nonlinear payoff, convex optimization

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Chain Rule

BEGINNER

**ONE-SENTENCE MEANING**

The chain rule differentiates a function inside another function.

**MEMORY JOGGER**

Outer change times inner change.

**PLAIN-ENGLISH CORE**

Quant pipelines are composed transformations: raw prices become returns, returns become features, features become signals, signals become positions, positions become PnL. The chain rule explains how sensitivity propagates through this stack.

**QUANT USE + MARKET EXAMPLE**

Quant use: Trace how a small data or parameter change moves through a model pipeline.

Example: If  $\text{signal} = \tanh(\text{zscore}(\text{momentum}))$ , sensitivity to momentum depends on both the z-score scaling and the tanh slope.

**FORMULA INTUITION**

If  $y = f(g(x))$ , then  $dy/dx = f'(g(x)) * g'(x)$ .

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

In system design, keep transformation stages explicit so sensitivity bugs can be isolated. Automatic differentiation libraries implement chain rule at scale.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: The chain rule is the math version of dependency tracing through a research pipeline.

Confusions to avoid: Do not differentiate only the final formula and forget hidden preprocessing transformations.

Related: composition, pipeline, automatic differentiation, gradients

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Product and Quotient Rules

BEGINNER

ONE-SENTENCE MEANING

These rules differentiate multiplied or divided functions.

MEMORY JOGGER

Products and ratios need special rules; do not just differentiate each piece independently.

PLAIN-ENGLISH CORE

Finance uses ratios constantly: return over risk, price over earnings, spread over volatility, and weighted products like position times return. When both numerator and denominator move, local sensitivity requires the correct rule.

QUANT USE + MARKET EXAMPLE

Quant use: Analyze Sharpe-like ratios, normalized features, and PnL expressions.  
Example:  $PnL = position(t) * return(t)$ . If position and return both vary, the local change has two moving parts.

DIAGNOSTIC MINI-TABLE

| Item              | Use               | Watch for              |
|-------------------|-------------------|------------------------|
| Raw change        | unit movement     | scale mismatch         |
| Return            | capital growth    | compounding convention |
| Derivative        | local sensitivity | smoothness assumption  |
| Finite difference | sampled estimate  | noise and alignment    |

FORMULA INTUITION

Product:  $(uv)' = u'v + uv'$ . Quotient:  $(u/v)' = (u'v - uv') / v^2$ .

PYTHON / SYSTEM IMPLEMENTATION IDEA

When optimizing ratio metrics, handle unstable denominators with floors, regularization, or alternative objectives.

SYSTEM FIT, CONFUSIONS, RELATED TERMS

System fit: Ratio sensitivity appears in objective functions, diagnostics, and normalized features.  
Confusions to avoid: A high ratio can be caused by a tiny denominator, not a strong numerator.  
Related: ratios, Sharpe, normalization, objective functions

QUICK RECALL

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Log Derivative and Growth Rate

BEGINNER

**ONE-SENTENCE MEANING**

The derivative of log value measures proportional growth rate.

**MEMORY JOGGER**

Log turns multiplicative growth into additive change.

**PLAIN-ENGLISH CORE**

Because capital compounds, proportional growth is often more meaningful than raw change. The derivative of  $\log(V)$  gives an instantaneous relative growth rate, which connects calculus to log returns and continuous compounding.

**QUANT USE + MARKET EXAMPLE**

Quant use: Use log returns and log equity curves to analyze compound growth and additive return decomposition.

Example: If log portfolio value rises steadily, the portfolio is compounding at a roughly stable proportional rate.

**FORMULA INTUITION**

$d/dt \log(V(t)) = V'(t) / V(t)$ . This is the instantaneous growth rate when  $V$  is positive.

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Store log returns for aggregation and cumulative log equity for diagnostics. Convert back with  $\exp(\text{cumulative\_log\_return})$ .

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Log derivative thinking connects calculus, compounding, and portfolio performance analysis.

Confusions to avoid: Log returns require positive prices or values; they do not handle zero or negative values directly.

Related: log returns, continuous compounding, growth rate, exponential function

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Elasticity

BEGINNER

**ONE-SENTENCE MEANING**

Elasticity measures percent output change per percent input change.

**MEMORY JOGGER**

Sensitivity in percentage terms.

**PLAIN-ENGLISH CORE**

Elasticity is useful when raw units are misleading. It asks: if input changes by 1 percent, approximately how many percent does output change? This is derivative thinking scaled by input and output levels.

**QUANT USE + MARKET EXAMPLE**

Quant use: Compare sensitivity across assets, features, or models with different units.

Example: A demand model, execution cost model, or leverage rule can use elasticity to measure proportional response.

**FORMULA INTUITION**

Elasticity =  $(dy/dx) * (x/y)$ , assuming  $x$  and  $y$  are positive and differentiable.

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Estimate elasticity by regressing  $\log(y)$  on  $\log(x)$  when the relationship is approximately power-law and positive.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Elasticity belongs in normalized sensitivity diagnostics and economic interpretation.

Confusions to avoid: Elasticity is local unless estimated across a range; it can change with regime.

Related: log derivative, scaling, sensitivity, normalization

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Partial Derivatives

BEGINNER

**ONE-SENTENCE MEANING**

A partial derivative measures sensitivity to one input while holding other inputs fixed.

**MEMORY JOGGER**

Change one knob, freeze the others.

**PLAIN-ENGLISH CORE**

Most market models have multiple inputs: price, volatility, time, interest rates, weights, parameters, and features. Partial derivatives isolate one dimension of local response.

**QUANT USE + MARKET EXAMPLE**

Quant use: Interpret option Greeks, risk contributions, and model parameter sensitivities.

Example: Option delta is a partial derivative of option value with respect to underlying price, holding other inputs fixed locally.

**FORMULA INTUITION**

Partial derivative:  $\partial f / \partial x_i$ . It asks how  $f$  changes when  $x_i$  changes and other inputs are held constant.

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Use scenario analysis: bump one input at a time, recompute output, and record the marginal effect.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Partial derivatives are the workhorse of sensitivity dashboards and risk explainers.

Confusions to avoid: Holding other inputs fixed may be unrealistic if variables co-move in real markets.

Related: gradient, Greeks, sensitivity analysis, scenario testing

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Gradient Vector

BEGINNER

**ONE-SENTENCE MEANING**

The gradient collects all partial derivatives into one vector.

**MEMORY JOGGER**

A compass pointing toward steepest increase.

**PLAIN-ENGLISH CORE**

For a multivariable function, the gradient tells which direction in input space increases the output fastest locally. Optimizers use gradients to move parameters; risk systems use gradients to summarize exposure.

**QUANT USE + MARKET EXAMPLE**

Quant use: Optimize models, measure portfolio exposure, and diagnose which input matters most.

Example: A loss function gradient tells how to adjust each model parameter to reduce error.

**FORMULA INTUITION**

$\text{grad } f = [\text{partial } f/\text{partial } x_1, \text{partial } f/\text{partial } x_2, \dots, \text{partial } f/\text{partial } x_n]$ .

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

For numeric gradients, bump each parameter independently or use autograd frameworks when the model is differentiable.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Gradients connect model training, risk decomposition, and parameter diagnostics.

Confusions to avoid: The gradient is local; following it blindly can still land in a poor region of a noisy objective surface.

Related: partial derivative, optimization, gradient descent, exposure

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Multivariable Market Functions

BEGINNER

**ONE-SENTENCE MEANING**

A multivariable function depends on more than one input.

**MEMORY JOGGER**

Markets are multi-knob systems.

**PLAIN-ENGLISH CORE**

A single output often depends on many drivers. Portfolio return depends on weights and asset returns. Option value depends on underlying price, volatility, time, rates, and strike. A backtest score depends on parameters, costs, data sample, and rules.

**QUANT USE + MARKET EXAMPLE**

Quant use: Model systems where changing one input may interact with other inputs.

Example: Sharpe(params, cost, regime) is a multivariable score, not a single-variable object.

**FORMULA INTUITION**

$y = f(x_1, x_2, \dots, x_n)$ . Sensitivity can be measured one input at a time or by directional changes.

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Represent parameter grids as structured arrays and evaluate surfaces, not isolated points.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Multivariable thinking prevents overconfidence from one-dimensional parameter sweeps.

Confusions to avoid: A parameter that looks good alone may fail when another parameter changes.

Related: partial derivatives, gradient, Hessian, heatmap

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.



# Jacobian for Feature Transforms

BEGINNER

## ONE-SENTENCE MEANING

A Jacobian is a matrix of partial derivatives for vector-valued functions.

## MEMORY JOGGER

Many outputs, many inputs, organized as a sensitivity table.

## PLAIN-ENGLISH CORE

Feature pipelines often map many raw inputs to many transformed outputs. The Jacobian describes how each output responds to each input locally. It is the multivariable extension of a derivative when outputs are vectors.

## QUANT USE + MARKET EXAMPLE

Quant use: Understand transformation sensitivity, factor models, and multi-output risk systems.

Example: A feature vector built from returns, volatility, and volume has sensitivity to each raw data stream.

## FORMULA INTUITION

$J_{ij} = \text{partial output}_i / \text{partial input}_j$ . Rows are outputs; columns are inputs.

## PYTHON / SYSTEM IMPLEMENTATION IDEA

In code, approximate the Jacobian by bumping each input column and recording changes in all output columns.

## SYSTEM FIT, CONFUSIONS, RELATED TERMS

System fit: Jacobians are useful for pipeline auditing and model explanation when many features are derived from shared inputs.

Confusions to avoid: A Jacobian can be large; summarize it with norms, heatmaps, or top sensitivities.

Related: gradient, matrix, feature pipeline, sensitivity map

## QUICK RECALL

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Hessian Matrix

BEGINNER

**ONE-SENTENCE MEANING**

The Hessian collects second partial derivatives and describes curvature in many dimensions.

**MEMORY JOGGER**

Gradient tells slope; Hessian tells how the slope bends.

**PLAIN-ENGLISH CORE**

Optimization problems depend heavily on curvature. A flat surface, sharp valley, and unstable ridge can all have different Hessian behavior. In portfolio and model optimization, curvature affects stability and confidence.

**QUANT USE + MARKET EXAMPLE**

Quant use: Diagnose parameter stability, convexity, and optimizer behavior.

Example: A broad plateau in a parameter surface suggests less fragile tuning than a narrow spike.

**FORMULA INTUITION**

$H_{ij} = \partial^2 f / \partial x_i \partial x_j$ . Diagonal terms are own-curvatures; off-diagonal terms are interactions.

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Use parameter heatmaps and local perturbation tests as practical Hessian intuition when exact formulas are unavailable.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Hessian thinking belongs in optimization diagnostics and robustness analysis.

Confusions to avoid: A precise optimum on a noisy backtest surface can be mathematically impressive and practically useless.

Related: second derivative, curvature, convexity, optimization

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Optimization: Choosing Inputs to Improve an Objective

## ONE-SENTENCE MEANING

Optimization searches for inputs that maximize or minimize a function.

## MEMORY JOGGER

Pick the knobs that make the score better, but verify the score is real.

## PLAIN-ENGLISH CORE

Many quant tasks are optimization tasks: choose parameters, choose portfolio weights, minimize forecast error, maximize utility, or reduce risk. Calculus gives conditions and algorithms for finding better values.

## QUANT USE + MARKET EXAMPLE

Quant use: Formalize model fitting, portfolio construction, and parameter selection.

Example: Choose moving-average lookbacks that maximize validation performance, not just in-sample performance.

## FORMULA INTUITION

Minimize  $J(\theta)$  or maximize  $S(\theta)$ . At an interior smooth optimum, gradient is often zero or near zero.

## PYTHON / SYSTEM IMPLEMENTATION IDEA

Separate training, validation, and test periods. Optimization without out-of-sample validation is curve fitting.

## SYSTEM FIT, CONFUSIONS, RELATED TERMS

System fit: Optimization is part of research, but validation is what determines whether it belongs in deployment.

Confusions to avoid: Best in sample does not mean best future performance.

Related: objective function, gradient, overfitting, validation

## QUICK RECALL

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Critical Points

BEGINNER

**ONE-SENTENCE MEANING**

Critical points are locations where a derivative is zero or undefined.

**MEMORY JOGGER**

Flat spots, corners, or problem points.

**PLAIN-ENGLISH CORE**

In simple calculus, maxima and minima often occur at critical points. In market modeling, a flat derivative can indicate an optimum, a plateau, saturation, or a dead model response.

**QUANT USE + MARKET EXAMPLE**

Quant use: Investigate candidate optima and saturation zones in objective surfaces.

Example: A signal transform like  $\tanh$  can saturate, making the derivative near zero for extreme inputs.

**FORMULA INTUITION**

Critical condition:  $f'(x) = 0$  or  $f'(x)$  does not exist. For many variables:  $\text{grad } f = 0$ .

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Do not only locate critical points. Check neighboring values, regimes, and out-of-sample stability.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Critical-point thinking supports parameter tuning and model response diagnostics.

Confusions to avoid: A zero derivative can be a maximum, minimum, saddle, or flat region.

Related: first derivative test, second derivative test, saddle point, plateau

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Taylor Approximation

BEGINNER

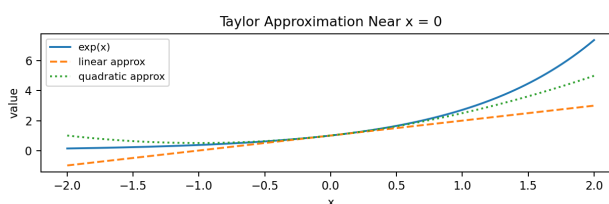
## ONE-SENTENCE MEANING

Taylor approximation replaces a function near a point with a polynomial using derivatives.

## MEMORY JOGGER

Use slope and curvature to make a local shortcut.

## APPROXIMATION CHART



The linear approximation is simple; the quadratic approximation adds curvature and usually works better near the expansion point.

## PLAIN-ENGLISH CORE

Taylor expansions are the foundation of many approximations in finance. They explain why small moves can be estimated with first-order sensitivity and why larger moves need curvature terms.

## QUANT USE + MARKET EXAMPLE

Quant use: Approximate option PnL, risk changes, and nonlinear model responses locally.

Example: Option PnL can be approximated by  $\Delta S + 0.5 \Gamma (\Delta S)^2$  for small underlying moves.

## FORMULA INTUITION

$f(x+h) \sim f(x) + f'(x)h + 0.5 f''(x)h^2$ . Higher terms improve local accuracy when valid.

## PYTHON / SYSTEM IMPLEMENTATION IDEA

Build scenario reports with first-order and second-order approximations, then compare against full repricing.

## SYSTEM FIT, CONFUSIONS, RELATED TERMS

System fit: Taylor thinking connects derivatives to practical what-if analysis.

Confusions to avoid: Taylor approximations are local. Far from the expansion point, they can break badly.

Related: linearization, gamma, convexity, local model

## QUICK RECALL

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Linearization

BEGINNER

**ONE-SENTENCE MEANING**

Linearization approximates a curved function with a tangent line near a point.

**MEMORY JOGGER**

Pretend the curve is straight for a tiny neighborhood.

**PLAIN-ENGLISH CORE**

Linearization is a first-order approximation. It is fast and interpretable, but it ignores curvature. In finance, many risk reports are first-order because they are easy to compute and explain.

**QUANT USE + MARKET EXAMPLE**

Quant use: Estimate small shock effects quickly using local sensitivities.

Example: Portfolio change  $\sim$  sum of exposures times factor shocks.

**FORMULA INTUITION**

$f(x+h) \sim f(x) + f'(x)h$ . Multivariable:  $f(x+dx) \sim f(x) + \text{grad } f \cdot dx$ .

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Use linear approximations for quick risk checks, then stress test with full nonlinear repricing.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Linearization is useful for monitoring, not sufficient for all risk control.

Confusions to avoid: Linear risk can underestimate loss when payoff curvature is large.

Related: Taylor approximation, gradient, delta, factor exposure

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Quadratic Approximation

BEGINNER

**ONE-SENTENCE MEANING**

A quadratic approximation adds curvature to the linear approximation.

**MEMORY JOGGER**

Line plus bend.

**PLAIN-ENGLISH CORE**

Second-order terms matter when moves are not tiny or the payoff is nonlinear. In options, gamma is the classic second-order term. In optimization, quadratic approximations describe local curvature around an optimum.

**QUANT USE + MARKET EXAMPLE**

Quant use: Improve local risk estimates and understand optimizer curvature.

Example: A delta-gamma approximation estimates option PnL better than delta alone when the underlying move is moderate.

**FORMULA INTUITION**
$$f(x+h) \sim f(x) + f'(x)h + 0.5 f''(x)h^2.$$
**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Compare first-order and second-order estimates against actual recomputed values across shock sizes.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Quadratic approximation appears in scenario analysis, option risk, and second-order optimization.

Confusions to avoid: Second-order does not guarantee accuracy for jumps, discontinuities, or very large moves.

Related: Taylor series, gamma, Hessian, convexity

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Integration as Accumulation

BEGINNER

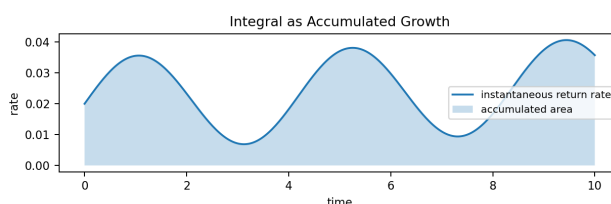
## ONE-SENTENCE MEANING

An integral adds up infinitely many tiny pieces.

## MEMORY JOGGER

Area under the curve equals accumulated total.

## AREA CHART



Accumulation can be read as area under a rate curve. Discrete data approximates this with sums.

## PLAIN-ENGLISH CORE

If derivatives measure local change, integrals measure accumulated effect. In finance, integration appears in cumulative return, total exposure over time, expected value, and continuous compounding.

## QUANT USE + MARKET EXAMPLE

Quant use: Turn rates into totals and densities into probabilities or expectations.

Example: If  $r(t)$  is an instantaneous return rate, the integral of  $r(t)$  over time gives cumulative log growth.

## FORMULA INTUITION

Integral from  $a$  to  $b$  of  $f(t)$  dt. Interpret as accumulated area or total effect across the interval.

## PYTHON / SYSTEM IMPLEMENTATION IDEA

Approximate integrals with sums over sampled data: `total = sum(values * delta_t)`.

## SYSTEM FIT, CONFUSIONS, RELATED TERMS

System fit: Integration links time-local rates to full-period performance and risk measures.

Confusions to avoid: Adding raw rates requires consistent time units and sampling intervals.

Related: Riemann sum, cumulative return, expectation, area under curve

## QUICK RECALL

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.



# Riemann Sum

BEGINNER

**ONE-SENTENCE MEANING**

A Riemann sum approximates an integral by adding rectangles.

**MEMORY JOGGER**

Break the area into bars, add the bars.

**PLAIN-ENGLISH CORE**

Market data is sampled discretely, so real calculations are often sums, not perfect integrals. A Riemann sum is the bridge from discrete observations to continuous accumulation.

**QUANT USE + MARKET EXAMPLE**

Quant use: Approximate total exposure, cumulative returns, realized variance, and area-style metrics.  
Example: Realized variance is often computed by summing squared returns over a period.

**FORMULA INTUITION**

$\text{Integral } f(t)dt \sim \text{sum } f(t_i) * \text{delta}_t$ . Smaller intervals can improve approximation if data quality supports it.

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Use explicit time deltas when bars are irregular. Do not assume equal spacing if timestamps have gaps.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Riemann-sum thinking belongs in aggregation logic for returns, volatility, and exposure.  
Confusions to avoid: More frequent sampling can add microstructure noise instead of accuracy.  
Related: integral, discrete data, realized variance, sampling

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Definite Integral

BEGINNER

**ONE-SENTENCE MEANING**

A definite integral accumulates a function over a specific interval.

**MEMORY JOGGER**

From start to end, how much total area?

**PLAIN-ENGLISH CORE**

A definite integral has bounds, which makes it operational. It asks for the total contribution from time a to time b, price level A to B, or state range lower to upper.

**QUANT USE + MARKET EXAMPLE**

Quant use: Compute cumulative quantities over backtest windows, holding periods, or distribution ranges.

Example: Expected payoff integrates payoff(x) times probability density(x) over possible future prices.

**FORMULA INTUITION**

$\int_a^b f(x) dx$ . The bounds a and b define the accumulation window.

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

In backtests, make the window explicit: start date, end date, inclusivity rule, timezone, and missing data policy.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Definite integrals map naturally to windowed analytics and expectation calculations.

Confusions to avoid: Changing bounds can change conclusions; avoid cherry-picked windows.

Related: windowing, expectation, cumulative metrics, bounds

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Integral of a Return Rate

BEGINNER

## ONE-SENTENCE MEANING

Integrating an instantaneous return rate gives cumulative log growth.

## MEMORY JOGGER

Rate over time becomes total growth.

## PLAIN-ENGLISH CORE

Continuous compounding models describe growth with an instantaneous rate  $r(t)$ . Add those tiny rates through time and exponentiate to recover value growth. This is the continuous-time cousin of summing log returns.

## QUANT USE + MARKET EXAMPLE

Quant use: Connect log returns, compounding, and continuous-time models.

Example: If  $r(t)$  is constant at 0.05 per year, one year of growth gives  $\exp(0.05)-1$  approximately 5.13 percent.

## FORMULA INTUITION

$V(T) = V(0) * \exp(\int_0^T r(t) dt)$ . If  $r$  is constant:  $V(T) = V(0) \exp(rT)$ .

## PYTHON / SYSTEM IMPLEMENTATION IDEA

For discrete data, `cumulative_log = log_returns.cumsum()`; `equity = initial * exp(cumulative_log)`.

## SYSTEM FIT, CONFUSIONS, RELATED TERMS

System fit: This is the foundation for continuous compounding and log-equity analytics.

Confusions to avoid: Simple returns compound by multiplication; log returns aggregate by addition.

Related: log return, exponential growth, continuous compounding, integral

## QUICK RECALL

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Area Under Curve Metrics

BEGINNER

**ONE-SENTENCE MEANING**

Area-style metrics summarize total exposure or total error across a range.

**MEMORY JOGGER**

Not just height; width matters too.

**PLAIN-ENGLISH CORE**

A model can have moderate values for a long time or extreme values briefly. Integration-like metrics capture total accumulated effect. This is useful for exposure, turnover, cost, drawdown duration, and error.

**QUANT USE + MARKET EXAMPLE**

Quant use: Measure accumulated risk, accumulated cost, and time spent in adverse states.

Example: A shallow drawdown lasting 200 days may be psychologically and operationally worse than a sharp 2-day dip.

**FORMULA INTUITION**

Total effect  $\sim \sum \text{value}_t * \text{delta}_t$ . For equal bars,  $\text{delta}_t$  is often treated as 1 bar.

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Track time-under-water, cumulative transaction cost, cumulative absolute exposure, and integrated error.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Area metrics complement point metrics like max drawdown or peak exposure.

Confusions to avoid: A maximum value ignores duration; area helps capture persistence.

Related: integral, duration, exposure, cumulative cost

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Fundamental Theorem of Calculus

BEGINNER

**ONE-SENTENCE MEANING**

Differentiation and integration are inverse operations under suitable conditions.

**MEMORY JOGGER**

Slope and accumulation are two sides of the same machine.

**PLAIN-ENGLISH CORE**

The theorem connects local change to total change. If you know the rate of change, you can integrate it to recover accumulated change. If you know the accumulated function, you can differentiate it to recover the rate.

**QUANT USE + MARKET EXAMPLE**

Quant use: Connect return rates to cumulative growth and cumulative PnL to incremental PnL.

Example: Daily PnL increments accumulate into an equity curve; differences of the equity curve recover period PnL.

**FORMULA INTUITION**

If  $F'(x)=f(x)$ , then  $\int_a^b f(x)dx = F(b)-F(a)$ .

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Validate aggregation pipelines by checking that cumulative sums and differences invert each other except for starting value and missing data.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: This theorem is a sanity-check principle for cumulative analytics.

Confusions to avoid: Transaction costs, cash flows, and corporate actions can break naive inversion.

Related: derivative, integral, cumulative PnL, returns

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Continuous Compounding Differential Equation

BEGINNER

**ONE-SENTENCE MEANING**

Continuous compounding models value growth with  $dV/dt = rV$ .

**MEMORY JOGGER**

Growth rate times current value equals value change.

**PLAIN-ENGLISH CORE**

The equation says the rate of dollar change is proportional to the current value. This creates exponential growth and is the continuous form behind many compounding formulas.

**QUANT USE + MARKET EXAMPLE**

Quant use: Understand log returns, discounting, and continuous-time asset models.

Example: A bank account growing at constant continuous rate  $r$  follows  $V(t)=V(0)\exp(rt)$ .

**FORMULA INTUITION**

$dV/dt = rV$ . Solution for constant  $r$ :  $V(t)=V(0)e^{(rt)}$ .

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Use  $\exp(r*dt)$  for continuous compounding over interval  $dt$ . Be explicit whether  $r$  is annualized and what  $dt$  units use.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Continuous compounding is a base assumption in many pricing and growth models.

Confusions to avoid: APR, simple rate, periodic compounding, and continuous rate are not identical.

Related: exponential function, log return, differential equation, discounting

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Differential Equations

BEGINNER

**ONE-SENTENCE MEANING**

A differential equation defines a function through its rate of change.

**MEMORY JOGGER**

Instead of saying where the curve is, say how it moves.

**PLAIN-ENGLISH CORE**

Many financial models specify dynamics rather than direct formulas. They describe how price, volatility, rates, or wealth evolve over time. Solving or simulating the equation gives the path.

**QUANT USE + MARKET EXAMPLE**

Quant use: Read continuous-time models and understand simulated market dynamics.

Example: A mean-reverting process can define the change in a variable as proportional to the gap from its long-run mean.

**FORMULA INTUITION**

Generic form:  $dx/dt = f(x,t)$ . The next movement depends on current state  $x$  and time  $t$ .

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Simulate simple equations by stepping forward:  $x_{\text{next}} = x + f(x,t) \cdot dt$ . Use smaller  $dt$  for accuracy but watch noise and compute cost.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Differential equations connect model assumptions to generated paths and scenario analysis.

Confusions to avoid: A differential equation is a model assumption, not proof that markets follow that law.

Related: ODE, SDE, simulation, mean reversion

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Geometric Brownian Motion Structure

BEGINNER

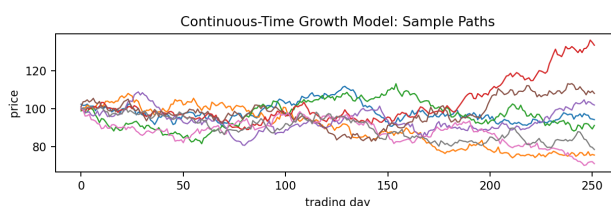
## ONE-SENTENCE MEANING

GBM is a continuous-time model where log price has drift and random diffusion.

## MEMORY JOGGER

Trend plus noise, compounded continuously.

## MONTE CARLO CHART



Sample paths show the range of outcomes produced by drift plus random diffusion assumptions.

## PLAIN-ENGLISH CORE

Geometric Brownian motion is a foundational model in finance because it keeps prices positive and creates lognormal price distributions under its assumptions. It is not a complete market truth, but it is a core reference model.

## QUANT USE + MARKET EXAMPLE

Quant use: Understand option pricing intuition, Monte Carlo paths, and drift-volatility decomposition.

Example: A simulated equity path can use annual drift  $\mu$  and volatility  $\sigma$  to generate random compounded returns.

## FORMULA INTUITION

Informal:  $dS/S = \mu dt + \sigma dW$ . Drift is expected direction; diffusion is random shock.

## PYTHON / SYSTEM IMPLEMENTATION IDEA

Simulate log returns:  $(\mu - 0.5\sigma^2)dt + \sigma\sqrt{dt}\text{normal\_noise}$ , then exponentiate cumulative sums.

## SYSTEM FIT, CONFUSIONS, RELATED TERMS

System fit: GBM is a baseline model for scenario generation and pricing intuition.

Confusions to avoid: GBM assumes constant volatility and continuous paths; real markets have jumps, regimes, and volatility clustering.

Related: stochastic process, drift, diffusion, Brownian motion

## QUICK RECALL

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.



# Drift and Diffusion

BEGINNER

**ONE-SENTENCE MEANING**

Drift is the directional component; diffusion is the random shock component.

**MEMORY JOGGER**

Drift is the current; diffusion is the turbulence.

**PLAIN-ENGLISH CORE**

Continuous-time market models often separate predictable trend-like movement from random uncertainty. Drift is usually hard to estimate reliably. Diffusion, connected to volatility, is often more observable but still time-varying.

**QUANT USE + MARKET EXAMPLE**

Quant use: Separate expected return assumptions from risk/noise assumptions.

Example: A stock may have positive long-run drift but daily movement dominated by diffusion noise.

**FORMULA INTUITION**

$dS/S = \mu dt + \sigma dW$ .  $\mu dt$  is drift;  $\sigma dW$  is diffusion.

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

When simulating, stress test multiple drift values because drift estimation error can dominate long-horizon conclusions.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Drift and diffusion assumptions feed scenario generation, risk simulation, and pricing models.

Confusions to avoid: Historical average return is a noisy estimate of future drift.

Related: GBM, volatility, stochastic process, Monte Carlo

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Sensitivity Greeks: Delta

BEGINNER

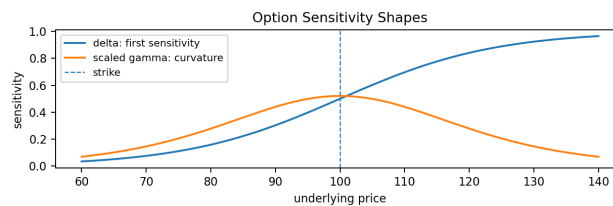
## ONE-SENTENCE MEANING

Delta measures option value sensitivity to the underlying price.

## MEMORY JOGGER

Underlying moves 1; option moves about delta, locally.

## GREEK SENSITIVITY CHART



Delta is first-order response; gamma is curvature response around the strike region.

## PLAIN-ENGLISH CORE

Delta is a partial derivative. It is local, changes as inputs change, and works best for small moves. Delta also serves as a hedge ratio in simplified option hedging.

## QUANT USE + MARKET EXAMPLE

Quant use: Measure first-order directional exposure in option portfolios.

Example: A call option with delta 0.55 gains roughly 0.55 for a 1 dollar underlying increase, before other effects.

## FORMULA INTUITION

Delta =  $\text{partial option\_value} / \text{partial underlying\_price}$ .

## PYTHON / SYSTEM IMPLEMENTATION IDEA

Compute delta using a pricing model or finite difference:  $\text{price}(S+h) - \text{price}(S-h)$  divided by  $2h$ .

## SYSTEM FIT, CONFUSIONS, RELATED TERMS

System fit: Delta belongs in risk dashboards, hedging logic, and scenario reports.

Confusions to avoid: Delta is not constant; it changes with price, time, volatility, and moneyness.

Related: partial derivative, gamma, Greeks, hedging

## QUICK RECALL

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Gamma and Convex Option Exposure

BEGINNER

**ONE-SENTENCE MEANING**

Gamma measures how delta changes as the underlying price changes.

**MEMORY JOGGER**

Gamma is delta acceleration.

**PLAIN-ENGLISH CORE**

Gamma is a second derivative. Positive gamma means directional exposure increases in favorable movement and decreases in unfavorable movement, before costs and other effects. It is central to nonlinear option risk.

**QUANT USE + MARKET EXAMPLE**

Quant use: Understand convexity, hedging error, and nonlinear PnL under larger moves.

Example: A long option position can have positive gamma, making its delta rise as the underlying rallies.

**FORMULA INTUITION**

$\text{Gamma} = \frac{\partial^2 \text{option\_value}}{\partial \text{underlying\_price}^2} = \frac{\partial \text{delta}}{\partial S}$ .

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Estimate gamma with a second finite difference:  $[V(S+h) - 2V(S) + V(S-h)] / h^2$ .

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Gamma belongs in second-order risk reports and stress testing.

Confusions to avoid: Positive gamma can still lose money through theta decay or volatility changes.

Related: delta, second derivative, convexity, theta

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Vega, Theta, and Rho

BEGINNER

**ONE-SENTENCE MEANING**

These Greeks measure sensitivity to volatility, time, and interest rates.

**MEMORY JOGGER**

Different knobs, different partial derivatives.

**PLAIN-ENGLISH CORE**

Options are multivariable functions. Underlying price is only one input. Vega, theta, and rho remind you that an option can lose money even if the underlying moves correctly, because other inputs changed.

**QUANT USE + MARKET EXAMPLE**

Quant use: Decompose option PnL and stress test non-price risk drivers.

Example: A long option can gain from rising implied volatility even if spot price barely moves.

**FORMULA INTUITION**

Vega = partial  $V$  / partial vol. Theta = partial  $V$  / partial time. Rho = partial  $V$  / partial rate.

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Build bump-and-reprice reports for each input. Keep bump sizes standardized and documented.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Multi-Greek reporting is a practical application of partial derivatives.

Confusions to avoid: Greeks depend on model assumptions and units; theta per day differs from theta per year.

Related: partial derivatives, option pricing, scenario analysis, risk

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Risk Exposure as a Gradient

BEGINNER

**ONE-SENTENCE MEANING**

A portfolio gradient summarizes how value changes under small factor moves.

**MEMORY JOGGER**

Exposure vector equals local sensitivity map.

**PLAIN-ENGLISH CORE**

If portfolio value depends on market factors, its gradient with respect to those factors is a vector of exposures. This gives first-order risk decomposition and helps explain PnL.

**QUANT USE + MARKET EXAMPLE**

Quant use: Measure factor exposure, hedge direction, and marginal risk drivers.

Example: A portfolio may have positive equity beta, negative duration exposure, and positive oil sensitivity.

**FORMULA INTUITION**

$dV \sim \text{grad } V \cdot dx$ . Each gradient component multiplies a small factor shock.

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Estimate factor gradients using regression, risk model exposures, or direct scenario bumps.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Gradient exposure maps belong in portfolio monitoring and risk attribution.

Confusions to avoid: Linear exposure misses nonlinear and jump risks.

Related: gradient, beta, factor model, linearization

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Marginal Contribution to Portfolio Risk

BEGINNER

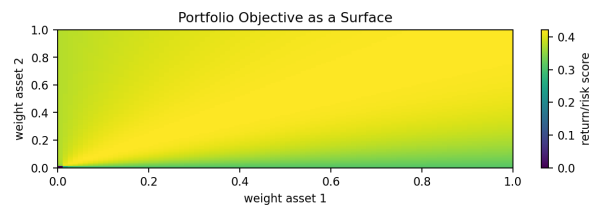
## ONE-SENTENCE MEANING

Marginal risk contribution measures how portfolio risk changes when a weight changes.

## MEMORY JOGGER

What does one more unit of this asset do to total risk?

## RISK SURFACE HEATMAP



Portfolio objectives can be surfaces over weights; calculus studies slope and curvature on that surface.

## PLAIN-ENGLISH CORE

Portfolio volatility is a function of weights and covariance. Calculus lets you ask how total risk responds to a small increase in one position. This supports risk budgeting.

## QUANT USE + MARKET EXAMPLE

Quant use: Allocate risk, not just capital, across positions.

Example: An asset with low standalone volatility can still add risk if it is highly correlated with the rest of the portfolio.

## FORMULA INTUITION

Portfolio variance =  $w^T \Sigma w$ . Gradient with respect to  $w$  is  $2 \Sigma w$ .

## PYTHON / SYSTEM IMPLEMENTATION IDEA

Compute covariance matrix, portfolio weights, and marginal risk contributions. Validate covariance stability across regimes.

## SYSTEM FIT, CONFUSIONS, RELATED TERMS

System fit: Marginal risk belongs in portfolio construction and risk budgeting modules.

Confusions to avoid: Risk contribution estimates are only as good as the covariance estimate.

Related: covariance, gradient, portfolio variance, risk parity

## QUICK RECALL

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Loss Functions

BEGINNER

## ONE-SENTENCE MEANING

A loss function converts model error into a score to minimize.

## MEMORY JOGGER

The model trains by asking: how bad was that?

## PLAIN-ENGLISH CORE

In machine learning and calibration, the loss function defines what the optimizer cares about. Calculus supplies gradients for moving parameters in the direction that reduces loss.

## QUANT USE + MARKET EXAMPLE

Quant use: Fit predictive models, calibrate pricing models, and tune forecasting systems.

Example: Mean squared error penalizes large forecast errors more heavily than small errors.

## FORMULA INTUITION

$MSE(\theta) = \text{average of } (\text{prediction}_i(\theta) - \text{target}_i)^2$ . The gradient points toward parameter changes that reduce MSE locally.

## PYTHON / SYSTEM IMPLEMENTATION IDEA

Choose losses aligned with trading objective. Forecast accuracy and trading profitability are related but not identical.

## SYSTEM FIT, CONFUSIONS, RELATED TERMS

System fit: Loss functions sit inside model training and calibration pipelines.

Confusions to avoid: Optimizing the wrong loss can produce a statistically good but economically weak model.

Related: objective function, gradient descent, calibration, validation

## QUICK RECALL

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Gradient Descent

BEGINNER

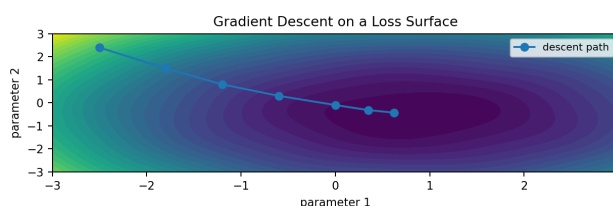
## ONE-SENTENCE MEANING

Gradient descent updates parameters by moving downhill on a loss surface.

## MEMORY JOGGER

Step opposite the slope.

## LOSS SURFACE CHART



Gradient descent follows local slope information across a parameter surface, but noise can distort the path.

## PLAIN-ENGLISH CORE

The gradient points toward steepest increase, so to minimize loss you move in the negative gradient direction. Step size matters: too small is slow; too large can overshoot or diverge.

## QUANT USE + MARKET EXAMPLE

Quant use: Train models, calibrate parameters, and understand optimizer behavior.

Example: A regression or neural model updates weights to reduce forecast error on training data.

## FORMULA INTUITION

$\theta_{\text{new}} = \theta_{\text{old}} - \text{learning\_rate} * \text{grad } J(\theta_{\text{old}})$ .

## PYTHON / SYSTEM IMPLEMENTATION IDEA

Track training loss, validation loss, gradient norms, and parameter drift. Use early stopping to reduce overfit.

## SYSTEM FIT, CONFUSIONS, RELATED TERMS

System fit: Gradient descent is a core engine inside model training systems.

Confusions to avoid: Lower training loss does not guarantee better out-of-sample trading performance.

Related: gradient, learning rate, optimization, overfitting

## QUICK RECALL

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.



# Learning Rate and Step Size

BEGINNER

**ONE-SENTENCE MEANING**

Step size controls how far an optimizer moves after reading the gradient.

**MEMORY JOGGER**

Tiny steps crawl; huge steps jump off the path.

**PLAIN-ENGLISH CORE**

Learning rate is an engineering choice with mathematical consequences. It shapes convergence speed, stability, and whether the optimizer bounces around or settles.

**QUANT USE + MARKET EXAMPLE**

Quant use: Tune model training stability and parameter search behavior.

Example: A model that oscillates between high and low loss may have a learning rate that is too large.

**FORMULA INTUITION**

Update size =  $\text{learning\_rate} * \text{gradient}$ . The same gradient causes different movement under different learning rates.

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Use learning-rate schedules, validation monitoring, and gradient clipping when needed.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Step-size control belongs in model-training infrastructure and experiment tracking.

Confusions to avoid: A stable optimizer can still overfit if the objective and validation setup are weak.

Related: gradient descent, convergence, optimizer, validation loss

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Numerical Derivatives

BEGINNER

ONE-SENTENCE MEANING

Numerical derivatives estimate slopes by evaluating nearby function values.

MEMORY JOGGER

Bump the input, recompute, compare.

PLAIN-ENGLISH CORE

Many quant systems contain functions without simple formulas: backtest engines, pricing libraries, transaction-cost models, or data transformations. Numerical derivatives approximate sensitivity by perturbation.

QUANT USE + MARKET EXAMPLE

Quant use: Measure local sensitivity when symbolic derivatives are unavailable.  
Example: Bump volatility from 20 percent to 20.1 percent and reprice an option to estimate vega.

DIAGNOSTIC MINI-TABLE

| Item              | Use               | Watch for              |
|-------------------|-------------------|------------------------|
| Raw change        | unit movement     | scale mismatch         |
| Return            | capital growth    | compounding convention |
| Derivative        | local sensitivity | smoothness assumption  |
| Finite difference | sampled estimate  | noise and alignment    |

FORMULA INTUITION

Forward:  $[f(x+h)-f(x)]/h$ . Central:  $[f(x+h)-f(x-h)]/(2h)$ .

PYTHON / SYSTEM IMPLEMENTATION IDEA

Try multiple h values. Too large gives approximation error; too small can create floating-point or simulation noise.

SYSTEM FIT, CONFUSIONS, RELATED TERMS

System fit: Numerical derivatives support black-box model diagnostics and scenario engines.  
Confusions to avoid: Noisy functions can produce unstable derivative estimates.  
Related: finite difference, sensitivity, bump-and-reprice, step size

QUICK RECALL

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Finite Differences

BEGINNER

**ONE-SENTENCE MEANING**

Finite differences approximate derivatives on discrete data.

**MEMORY JOGGER**

Use nearby sampled points instead of an ideal tangent.

**PLAIN-ENGLISH CORE**

Financial time series arrive as discrete observations. Finite differences are therefore the practical tool for estimating local change. They are simple but sensitive to sampling interval and noise.

**QUANT USE + MARKET EXAMPLE**

Quant use: Compute momentum, acceleration, realized variance, and numerical Greeks.  
Example: Daily return is a finite proportional difference between today and yesterday.

**FORMULA INTUITION**

First difference:  $x_t - x_{t-1}$ . Second difference:  $x_t - 2x_{t-1} + x_{t-2}$ .

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Use pandas `diff`, `pct_change`, and rolling smoothing. Always align timestamps to avoid lookahead.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Finite differences are core transformations in feature engineering.

Confusions to avoid: A difference over time  $t$  uses data from both endpoints; alignment matters for signals.

Related: difference quotient, derivative, discrete data, noise

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Discrete vs Continuous Models

ONE-SENTENCE MEANING

Discrete models step through time; continuous models define behavior at every instant.

MEMORY JOGGER

Bars versus flowing time.

PLAIN-ENGLISH CORE

Backtests are usually discrete because data is sampled. Many theoretical models are continuous because calculus becomes clean. Knowing the distinction prevents mismatched assumptions.

QUANT USE + MARKET EXAMPLE

Quant use: Translate theory into implementable backtest logic without pretending sampled data is continuous.  
Example: A continuous stop-loss rule may not execute at the theoretical level if the next real bar gaps beyond it.

DIAGNOSTIC MINI-TABLE

| Item              | Use               | Watch for              |
|-------------------|-------------------|------------------------|
| Raw change        | unit movement     | scale mismatch         |
| Return            | capital growth    | compounding convention |
| Derivative        | local sensitivity | smoothness assumption  |
| Finite difference | sampled estimate  | noise and alignment    |

FORMULA INTUITION

Discrete:  $x_{t+1}=x_t + \text{change}$ . Continuous:  $dx/dt=f(x,t)$  or  $dx = \dots$

PYTHON / SYSTEM IMPLEMENTATION IDEA

Make execution assumptions explicit: bar close, next open, intrabar fill, slippage, and latency.

SYSTEM FIT, CONFUSIONS, RELATED TERMS

System fit: Discrete/continuous translation belongs in backtest design and model implementation.  
Confusions to avoid: Continuous formulas can fail when execution and data are discrete.  
Related: sampling, backtesting, execution, differential equations

QUICK RECALL

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Smoothness Assumptions

BEGINNER

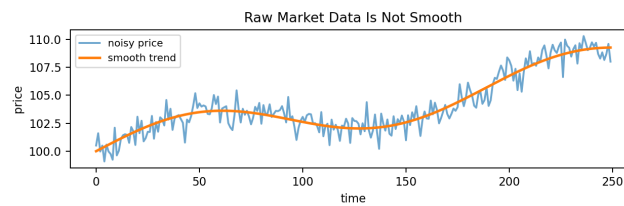
## ONE-SENTENCE MEANING

Smoothness means a function has well-behaved derivatives.

## MEMORY JOGGER

Calculus prefers clean curves; markets often give jagged data.

## RAW VS SMOOTH CHART



Smoothing can reveal slower structure, but it changes timing and can hide operational issues.

## PLAIN-ENGLISH CORE

Derivative-based reasoning assumes enough smoothness for local slopes to be meaningful. Raw price data can be noisy, discontinuous, or tick-constrained. Smooth transformations may be needed, but smoothing introduces lag.

## QUANT USE + MARKET EXAMPLE

Quant use: Decide when derivative-like indicators are meaningful and when they are just noise.

Example: A slope estimate on a 1-second crypto series may be dominated by microstructure noise.

## FORMULA INTUITION

Differentiability implies local linear approximation is valid:  $f(x+h) \sim f(x) + f'(x)h$  for small  $h$ .

## PYTHON / SYSTEM IMPLEMENTATION IDEA

Compare raw, smoothed, and volatility-normalized derivative features. Track signal decay and turnover.

## SYSTEM FIT, CONFUSIONS, RELATED TERMS

System fit: Smoothness assumptions belong in feature validation.

Confusions to avoid: Smoothing can make a signal look cleaner while delaying it or leaking future data if done incorrectly.

Related: noise, filtering, moving average, differentiability

## QUICK RECALL

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Non-Differentiable Market Features

BEGINNER

**ONE-SENTENCE MEANING**

Some functions have corners, jumps, or thresholds where derivatives fail or change abruptly.

**MEMORY JOGGER**

Corners break tangent-line thinking.

**PLAIN-ENGLISH CORE**

Trading rules often include max, min, stop-loss, absolute value, clipping, thresholds, and ranks. These can be non-differentiable at certain points. Optimizers and sensitivity reports need special care around those boundaries.

**QUANT USE + MARKET EXAMPLE**

Quant use: Recognize where model response changes abruptly due to rules, not market economics.

Example: A signal that goes long only when z-score  $> 2$  has a hard threshold at 2.

**FORMULA INTUITION**

A function can be continuous but not differentiable, like  $\text{abs}(x)$  at  $x=0$ .

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Use smoothed approximations for gradient-based optimization, or use search methods that do not require derivatives.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Non-differentiable rules belong in execution logic and model diagnostics.

Confusions to avoid: A backtest may look stable away from a threshold but flip behavior near the boundary.

Related: thresholds, clipping, ranks, discontinuity

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Noise and Derivatives

BEGINNER

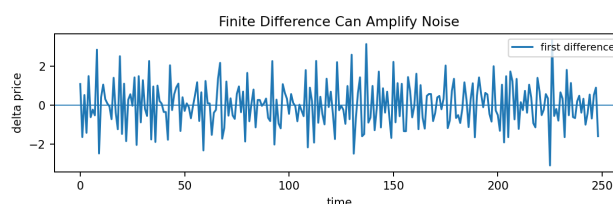
## ONE-SENTENCE MEANING

Derivative estimates can magnify high-frequency noise.

## MEMORY JOGGER

Taking differences turns small wiggles into loud signals.

## NOISE AMPLIFICATION CHART



Finite differences can turn small random wiggles into large noisy change estimates.

## PLAIN-ENGLISH CORE

A derivative measures change, so random small changes can dominate the estimate. This is a major practical issue for price data. Smoothing, longer horizons, and robust estimators help but each introduces tradeoffs.

## QUANT USE + MARKET EXAMPLE

Quant use: Avoid building signals that are just noise amplification.

Example: One-bar price differences may look predictive in sample but collapse after costs.

## FORMULA INTUITION

Finite difference derivative  $\sim [x_t - x_{t-1}] / dt$ . If  $dt$  is tiny, noise divided by  $dt$  can become huge.

## PYTHON / SYSTEM IMPLEMENTATION IDEA

Test derivative features after transaction costs and across sampling frequencies. Use walk-forward validation.

## SYSTEM FIT, CONFUSIONS, RELATED TERMS

System fit: Noise diagnostics belong in feature engineering and signal validation.

Confusions to avoid: More frequent data does not automatically mean better signal.

Related: finite differences, smoothing, microstructure noise, turnover

## QUICK RECALL

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Smoothing and Regularization

BEGINNER

**ONE-SENTENCE MEANING**

Smoothing reduces noise; regularization penalizes unstable or overly complex behavior.

**MEMORY JOGGER**

Trade sharpness for stability.

**PLAIN-ENGLISH CORE**

Derivative-like features often need smoothing. Model training often needs regularization. Both ideas reduce sensitivity to random variation, but both can also suppress real signals.

**QUANT USE + MARKET EXAMPLE**

Quant use: Improve robustness of features, models, and parameter estimates.

Example: A moving average slope is smoother than a one-day price difference but responds more slowly.

**FORMULA INTUITION**

Regularized objective: total loss = fit loss +  $\lambda$  \* penalty. The penalty discourages unstable parameters.

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Tune smoothing windows and regularization strength using validation data, not visual preference.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Smoothing and regularization are stability controls in the research system.

Confusions to avoid: A smoother backtest is not automatically more truthful; it may hide timing risk.

Related: moving averages, ridge, lasso, noise, stability

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.



# Indicator Derivatives

BEGINNER

**ONE-SENTENCE MEANING**

Indicator derivatives measure how technical indicators change through time.

**MEMORY JOGGER**

Not just indicator level; indicator slope.

**PLAIN-ENGLISH CORE**

A moving average, RSI, volatility estimate, or spread can have its own slope and curvature. These derivative-like features can describe transitions, but must be aligned and validated carefully.

**QUANT USE + MARKET EXAMPLE**

Quant use: Create features that capture regime shifts, trend changes, and volatility expansion.

Example: A rising realized volatility estimate can indicate a transition into a more volatile regime.

**FORMULA INTUITION**

Indicator slope  $\sim$   $\text{indicator}_t - \text{indicator}_{t-k}$  over  $k$  bars. Normalized slope divides by recent volatility or indicator scale.

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Generate indicator levels, slopes, and lagged slopes. Avoid using centered rolling windows that include future data.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Indicator derivatives are feature-engineering extensions for signal models.

Confusions to avoid: Indicator-on-indicator features can overfit quickly if not constrained.

Related: technical indicators, feature engineering, lagging, alignment

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Feature Engineering with Slopes

BEGINNER

**ONE-SENTENCE MEANING**

Slope features convert level data into direction and speed information.

**MEMORY JOGGER**

Levels say where; slopes say how it is moving.

**PLAIN-ENGLISH CORE**

Many features become more informative when converted into changes: price trend, volatility expansion, spread widening, volume acceleration, or factor momentum. The calculus analogy is useful, but the implementation is discrete.

**QUANT USE + MARKET EXAMPLE**

Quant use: Add dynamic information to model inputs without relying only on static levels.

Example: A credit spread level may be high, but a rapidly widening spread can carry different risk information.

**FORMULA INTUITION**

$\text{Slope}_k(x_t) = (x_t - x_{t-k}) / k$ . Vol-scaled slope = slope / rolling\_volatility.

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Create features at multiple horizons, then test incremental value and correlation with existing features.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Slope features belong in feature libraries with metadata for horizon, scaling, and alignment.

Confusions to avoid: More slope horizons can create multiple-testing risk and redundant features.

Related: momentum, finite difference, feature library, horizon

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Calibration Objective Functions

BEGINNER

**ONE-SENTENCE MEANING**

Calibration chooses model parameters so model outputs match observed targets.

**MEMORY JOGGER**

Make the model fit market observations, but do not worship the fit.

**PLAIN-ENGLISH CORE**

Pricing and risk models often have parameters that are chosen by minimizing pricing error. Calculus supports gradient-based calibration, while diagnostics check whether the fitted parameters are stable and meaningful.

**QUANT USE + MARKET EXAMPLE**

Quant use: Fit volatility surfaces, factor models, execution-cost models, and forecasting models.

Example: Implied volatility calibration minimizes differences between model option prices and market option prices.

**FORMULA INTUITION**

Objective =  $\sum_i \text{weight}_i * \text{error}_i(\theta)^2$ . Choose  $\theta$  to minimize objective.

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Track parameter time series after calibration. Unstable calibrated parameters may indicate model misspecification or bad data.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Calibration belongs in model fitting, but stability belongs in validation.

Confusions to avoid: A low calibration error can hide bad extrapolation outside observed instruments.

Related: optimization, loss function, gradients, validation

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Backtest Metrics as Functions

BEGINNER

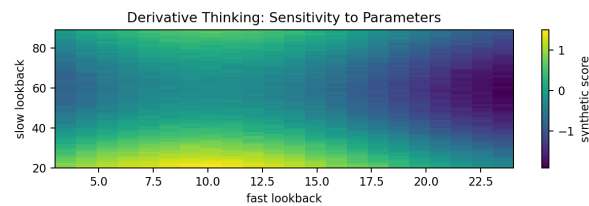
## ONE-SENTENCE MEANING

Backtest metrics are functions of data, rules, costs, and parameters.

## MEMORY JOGGER

Sharpe is not a number from heaven; it is output from a machine.

## PARAMETER SENSITIVITY HEATMAP



Stable regions are usually more credible than isolated hot spots in a parameter grid.

## PLAIN-ENGLISH CORE

Every backtest score depends on choices: universe, timeframe, costs, slippage, parameters, rebalancing, sizing, and filters. Treating metrics as functions makes sensitivity analysis natural.

## QUANT USE + MARKET EXAMPLE

Quant use: Diagnose whether performance survives small changes in assumptions.

Example:  $\text{Sharpe}(\text{parameter})$  can form a surface. A broad stable region is usually more credible than a narrow spike.

## FORMULA INTUITION

$\text{score} = F(\text{data, rules, costs, parameters})$ . Sensitivity asks how score changes when one input is perturbed.

## PYTHON / SYSTEM IMPLEMENTATION IDEA

Automate perturbation tests: cost up/down, delay fills, change lookbacks, change universe, shift dates.

## SYSTEM FIT, CONFUSIONS, RELATED TERMS

System fit: Metric-as-function thinking is central to robust backtest review.

Confusions to avoid: A single best metric value without sensitivity context is weak evidence.

Related: objective function, parameter surface, robustness, overfitting

## QUICK RECALL

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Drawdown as a Path Function

BEGINNER

**ONE-SENTENCE MEANING**

Drawdown depends on the running peak and the full path of returns.

**MEMORY JOGGER**

Loss from the high-water mark, not just one-period loss.

**PLAIN-ENGLISH CORE**

Drawdown is not a simple smooth function of current return. It depends on history through the running maximum. That path dependence makes derivative-style reasoning harder.

**QUANT USE + MARKET EXAMPLE**

Quant use: Understand path-dependent risk, recovery, and investor experience.

Example: Two strategies with the same final return can have very different drawdown paths.

**FORMULA INTUITION**

$\text{Drawdown}_t = \text{equity}_t / \text{running\_max}(\text{equity})_t - 1$ . The running max creates path dependence.

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Compute running peaks with cumulative maximum. Stress test order of returns, not just average return.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Path-risk metrics belong in backtest diagnostics and portfolio monitoring.

Confusions to avoid: Volatility and drawdown are related but not the same; drawdown is path dependent.

Related: running maximum, path dependence, risk, recovery

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Constraints in Optimization

**ONE-SENTENCE MEANING**

Constraints restrict which solutions an optimizer may choose.

**MEMORY JOGGER**

The best answer must still obey the rules.

**PLAIN-ENGLISH CORE**

Real portfolios have constraints: max leverage, long-only, sector caps, turnover limits, liquidity limits, and risk budgets. Calculus-based optimization must respect feasible regions.

**QUANT USE + MARKET EXAMPLE**

Quant use: Build implementable portfolios instead of mathematically convenient ones.  
Example: A portfolio optimizer may prefer 300 percent leverage, but a mandate may cap gross exposure at 100 percent.

**DIAGNOSTIC MINI-TABLE**

| Item              | Use               | Watch for              |
|-------------------|-------------------|------------------------|
| Raw change        | unit movement     | scale mismatch         |
| Return            | capital growth    | compounding convention |
| Derivative        | local sensitivity | smoothness assumption  |
| Finite difference | sampled estimate  | noise and alignment    |

**FORMULA INTUITION**

Optimize objective subject to constraints such as  $\text{sum weights} = 1$ ,  $\text{weights} \geq 0$ , or  $\text{risk} \leq \text{limit}$ .

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Use constrained optimizers and verify final weights against every operational rule.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Constraints connect mathematical optimization to deployable portfolio construction.  
Confusions to avoid: Unconstrained optima are often irrelevant in real trading systems.  
Related: portfolio optimization, feasible region, risk limits, Lagrange multipliers

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Lagrange Multiplier Intuition

BEGINNER

**ONE-SENTENCE MEANING**

Lagrange multipliers solve optimization problems with equality constraints.

**MEMORY JOGGER**

Optimize while staying on the allowed surface.

**PLAIN-ENGLISH CORE**

A constraint like fully invested weights changes the optimum. Lagrange multipliers introduce an extra variable that balances objective improvement against constraint satisfaction.

**QUANT USE + MARKET EXAMPLE**

Quant use: Understand constrained portfolio optimization and allocation problems.

Example: Maximize expected utility while requiring all portfolio weights to sum to one.

**FORMULA INTUITION**

Set  $\text{grad } f = \lambda \text{ grad } g$  for constraint  $g(x)=c$ . At optimum, objective gradient aligns with constraint gradient.

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

You do not need to derive every solver, but you should understand that constraints change first-order conditions.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: This is conceptual scaffolding for mean-variance optimization and risk budgeting.

Confusions to avoid: Lagrange methods assume smoothness and constraint regularity; real constraints can be inequalities or discrete.

Related: constraints, optimization, gradients, portfolio weights

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Expectation as an Integral

BEGINNER

**ONE-SENTENCE MEANING**

Expected value is a probability-weighted average, often written as an integral.

**MEMORY JOGGER**

Average over all possible outcomes, weighted by probability.

**PLAIN-ENGLISH CORE**

In continuous distributions, expectation integrates outcome times density. In simulations, you approximate this by averaging many sampled outcomes. This connects probability to calculus.

**QUANT USE + MARKET EXAMPLE**

Quant use: Price payoffs, evaluate strategy distributions, and compute risk estimates.

Example: Expected option payoff averages payoff across possible future prices under a probability model.

**FORMULA INTUITION**

$E[g(X)] = \int g(x) p(x) dx$ . Simulation approximation: average of  $g(\text{sample}_i)$ .

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Use Monte Carlo to approximate expectations when analytic integrals are hard. Track standard error.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Expectation integrals underlie pricing, risk, and simulation modules.

Confusions to avoid: Expected value is model-dependent and may not represent a typical single outcome.

Related: integral, probability density, Monte Carlo, payoff

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.



# Monte Carlo as Numerical Integration

BEGINNER

**ONE-SENTENCE MEANING**

Monte Carlo estimates integrals by random sampling.

**MEMORY JOGGER**

Average enough simulated worlds to estimate the weighted total.

**PLAIN-ENGLISH CORE**

When high-dimensional integrals are difficult, simulation becomes practical. You generate scenarios, compute outcomes, and average them. Accuracy improves with more samples, but slowly.

**QUANT USE + MARKET EXAMPLE**

Quant use: Estimate expected payoff, risk, drawdown distribution, and scenario outcomes.

Example: Simulate 10,000 price paths and average discounted option payoffs to estimate price.

**FORMULA INTUITION**

$E[g(X)] \sim (1/N) \sum g(X_i)$ . Standard error often falls like  $1/\sqrt{N}$ .

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Use fixed random seeds for reproducibility, vectorized simulations for speed, and confidence intervals for estimates.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: Monte Carlo belongs in pricing, risk, and stress-testing infrastructure.

Confusions to avoid: More simulations do not fix wrong model assumptions.

Related: expectation, integral, simulation, standard error

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# PDE Intuition for Pricing Models

BEGINNER

**ONE-SENTENCE MEANING**

A partial differential equation can describe how a price function changes across several variables.

**MEMORY JOGGER**

Many local sensitivity rules tied together into one equation.

**PLAIN-ENGLISH CORE**

Option pricing theory often leads to PDEs because option value depends on price, time, volatility assumptions, and hedging logic. Beginner goal: understand the role, not master derivation immediately.

**QUANT USE + MARKET EXAMPLE**

Quant use: Read the structure behind Black-Scholes-style pricing and finite-difference solvers.

Example: An option value surface changes across underlying price and time to expiry.

**FORMULA INTUITION**

A PDE links partial derivatives, such as time sensitivity, price sensitivity, and curvature, inside one equation.

**PYTHON / SYSTEM IMPLEMENTATION IDEA**

Numerical PDE solvers discretize price and time grids, then step backward from terminal payoff.

**SYSTEM FIT, CONFUSIONS, RELATED TERMS**

System fit: PDE intuition belongs in the advanced pricing track after derivatives and partial derivatives are comfortable.

Confusions to avoid: Knowing a PDE form does not mean its assumptions fit the market.

Related: partial derivative, option pricing, finite difference grid, Black-Scholes

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Research Pipeline: Calculus View

BEGINNER

## ONE-SENTENCE MEANING

The full research system can be viewed as chained functions and sensitivity checks.

## MEMORY JOGGER

Data -> features -> signal -> positions -> PnL -> diagnostics.

## PLAIN-ENGLISH CORE

Calculus thinking does not sit in one isolated math box. It appears when you transform data, measure local changes, optimize parameters, aggregate returns, and diagnose sensitivity.

## QUANT USE + MARKET EXAMPLE

Quant use: Build cleaner mental models for every research stage.

Example: A small transaction-cost change can propagate through score, parameter choice, and deployment decision.

## FORMULA INTUITION

$\text{score} = D(\text{PnL}(\text{signal}(\text{features}(\text{data})), \text{costs}, \text{constraints}))$ . Sensitivity can be checked at every nested layer.

## PYTHON / SYSTEM IMPLEMENTATION IDEA

Instrument pipelines with perturbation tests and metadata. Record how outputs respond to small changes in inputs and assumptions.

## SYSTEM FIT, CONFUSIONS, RELATED TERMS

System fit: This is the system-level map for using calculus in quant research.

Confusions to avoid: Do not only check final performance; check transformation behavior throughout the pipeline.

Related: chain rule, pipeline, sensitivity, validation

## QUICK RECALL

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Formula Summary Sheet

BEGINNER

## DERIVATIVE FAMILY

Difference quotient:  $[f(x+h)-f(x)]/h$ . Derivative: limit as  $h \rightarrow 0$ . Central finite difference:  $[f(x+h)-f(x-h)]/(2h)$ . Second finite difference:  $f(x+h)-2f(x)+f(x-h)$ .

## INTEGRAL FAMILY

Integral accumulates area or total effect. Discrete approximation:  $\sum f(t_i) \cdot \Delta t$ . Continuous compounding:  $V(T) = V(0) \cdot \exp(\int_0^T r(t) dt)$ .

## MULTIVARIABLE FAMILY

Partial derivative changes one input. Gradient collects all partials. Hessian collects second partials. Linear approximation:  $f(x+dx) \sim f(x) + \text{grad } f \cdot dx$ .

## OPTIMIZATION FAMILY

Objective: minimize  $J(\theta)$ . Gradient descent:  $\theta_{\text{new}} = \theta_{\text{old}} - \text{learning\_rate} \cdot \text{grad } J(\theta)$ . Constrained optimization adds feasible-region rules.

## FINANCE FAMILY

$\Delta = \partial V / \partial S$ .  $\Gamma = \partial^2 V / \partial S^2$ . Portfolio variance =  $w^T \Sigma w$ . Marginal variance gradient =  $2 \Sigma w$ .

## RECALL RULE

Before using a formula, state the units, assumptions, sampling interval, and whether the output is local, accumulated, or optimized.

## QUICK RECALL

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Failure Modes and Diagnostics

BEGINNER

## PURPOSE

Use this page when a calculus-inspired feature or model looks good but behaves strangely. Most failures come from noise, units, smoothness assumptions, overfitting, or local approximations being used outside their valid neighborhood.

## FAILURE-MODE TABLE

| Item                | Use                       | Watch for               |
|---------------------|---------------------------|-------------------------|
| Noisy derivative    | Signal flips constantly   | smooth or widen horizon |
| Wrong units         | Annual/day mixup          | standardize dt          |
| Overfit optimum     | narrow hot spot           | walk-forward tests      |
| Jump ignored        | model underestimates loss | gap stress test         |
| Tiny denominator    | ratio explodes            | floor or regularize     |
| Lookahead smoothing | too-good backtest         | causal windows only     |
| Local approximation | fails on large moves      | full scenario repricing |
| Covariance drift    | risk budget unstable      | regime checks           |

## DEBUGGING SEQUENCE

1. Check data alignment. 2. Check units and annualization. 3. Plot the function or surface. 4. Bump inputs. 5. Compare first-order approximation to full recomputation. 6. Validate across time, assets, regimes, and cost assumptions.

## RESEARCH INTERPRETATION

A stable calculus diagnostic should be explainable, reproducible, and robust to small implementation changes. If the result only exists under one fragile setting, treat it as a hypothesis, not a finding.

## VALIDATION QUESTIONS

Did the result survive different sampling intervals, trading costs, date ranges, universes, and nearby parameters? Did the same idea work after removing obvious outliers and after including realistic execution delay?

## WHAT TO LOG

Log input version, cleaning rules, parameter values, timestamp alignment, annualization convention, cost assumption, random seed, and full output path. Diagnostics are only useful if the exact calculation can be reproduced.

# Quick Recall Questions

BEGINNER

**DERIVATIVES**

What is the input? What is the output? Is the derivative local or averaged? Is the data smooth enough for slope to mean anything? What are the units of the derivative?

**INTEGRALS**

What is being accumulated? Over what bounds? Are the observations evenly spaced? Is the integral approximated by a sum? Does duration matter as much as magnitude?

**OPTIMIZATION**

What objective is being optimized? Does it match trading value? Is the optimum broad or narrow? Does it survive out-of-sample validation and cost stress?

**SENSITIVITY**

Which input is being bumped? Are other inputs truly fixed? Is the response linear, curved, discontinuous, or path-dependent?

**MARKET REALISM**

Are there gaps, jumps, tick-size effects, liquidity limits, slippage, threshold rules, or constraints that make the clean calculus story incomplete?

**IMPLEMENTATION**

Can the calculation be reproduced from raw data with no future leakage? Are parameters, timestamps, units, and conventions logged?

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.

# Index and Concept Links

BEGINNER

**CHANGE CONCEPTS**

Difference quotient, derivative, slope, rate of change, finite difference, momentum analogy, numerical derivatives, indicator derivatives, feature slopes.

**CURVATURE CONCEPTS**

Second derivative, acceleration, convexity, gamma, Hessian, quadratic approximation, nonlinear payoff, parameter surface stability.

**ACCUMULATION CONCEPTS**

Integral, Riemann sum, definite integral, return-rate integral, area metrics, fundamental theorem, expectation, Monte Carlo.

**OPTIMIZATION CONCEPTS**

Objective function, loss function, gradient, gradient descent, learning rate, critical points, constraints, Lagrange multipliers.

**MARKET-MODEL CONCEPTS**

Continuous compounding, differential equations, GBM, drift, diffusion, Greeks, risk gradient, marginal portfolio risk, PDE intuition.

**FINAL STUDY RULE**

When stuck, translate the concept into one sentence: this function takes input  $X$ , produces output  $Y$ , and I am measuring local change, accumulated effect, or best parameter choice.

**QUICK RECALL**

Ask: What is the input, what is the output, and what small change is being measured? Then ask whether the market data is smooth enough for that calculus idea to be trusted.